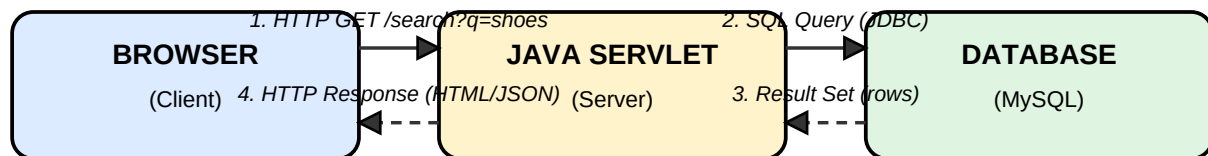


Web Application Basics: Client, Server & Database Assignment

Topics: HTTP requests, Java Servlets, static vs dynamic pages, and data flow

Q1. Flow Diagram — Flipkart-style Product Search

Below is a simple flow diagram showing how a product search request travels from the browser (client) to a Java Servlet (server), then to a Database, and back again.



1. User types 'shoes' and hits search -> browser sends an HTTP GET request to the servlet URL.
2. Servlet (doGet method) receives the request, extracts the search term, and queries the database using JDBC.
3. Database executes the SQL SELECT query and returns matching product rows to the servlet.
4. Servlet formats the data (HTML page or JSON) and sends it back as an HTTP response; browser renders the results.

Fig 1: Request/response flow between client, server (Java Servlet), and database.

Client: Browser — sends the request and displays the final page.

Server: Java Servlet — receives requests, applies logic, talks to the database.

Database: Stores product data and returns query results to the servlet.

Q2. Inspecting a Network Request (DevTools) — Zomato/Myntra

Steps followed: opened the site in a browser, pressed F12 (or right-click → Inspect) to open DevTools, clicked the **Network** tab, then typed a search term / applied a filter on the page. A typical request captured while searching for a dish/restaurant (e.g. searching "pizza" on Zomato, or filtering by size on Myntra) looks like this:

Field	Observed Value (example)
Request URL	https://www.zomato.com/webroutes/search/home?q=pizza&lat=..&lon=..
Request Method	GET
Status Code	200 OK
Response Type	JSON (application/json)
What it does	Sends the search keyword + location as query parameters; server returns a JSON list of matching restaurants

Note: The exact URL/parameters vary by session, login state, and app version — but the pattern (a GET request carrying the search text, returning JSON with status 200) is consistent across most search/filter actions on Zomato and Myntra.

Q3. Static HTML File — profile.html

A file named **profile.html** was created with a name, favorite app, and a short bio (see the attached file). When opened in a browser, it always displays the exact same content on every reload, because it is a **static page** — the browser simply reads and displays the fixed HTML file from disk/server; nothing changes unless the file itself is edited and saved again.

Observation: Refreshing the page 10 times, or opening it on different days, shows identical content. There is no server-side processing or database involved — the content is 'baked into' the file.

Q4. Static vs Dynamic Web Pages — 3 Differences

Aspect	Static Page (e.g. a basic About Us page)	Dynamic Page (e.g. Instagram)
Content source	Fixed HTML file; same content every time it is opened	Content is generated on the fly from a database (posts, likes, etc.)
Personalization	Same page shown to every visitor — no login, no user-specific data	Different for every user — your feed, profile, and notifications
How it updates	Only changes when a developer manually edits and uploads the file	Updates automatically in real time as new posts, comments, etc. are added

Q5. Data Flow — BookMyShow-style Ticket Booking

When a user books a movie ticket, the request travels through the client, server, and database roughly in these steps:

- **Step 1 — Client sends request:** The user selects seats and clicks 'Book Now'. The browser sends an HTTP POST request to the Java Servlet, containing details like movie ID, showtime, selected seats, and user info.
- **Step 2 — Servlet receives & validates:** The servlet's doPost() method reads the request data, checks seat availability and validity (e.g. seats not already booked).
- **Step 3 — Servlet talks to the database:** Using JDBC, the servlet runs SQL queries/updates — it checks the seats table and inserts a new row into the bookings table, and the database confirms the transaction.
- **Step 4 — Response sent back:** The servlet sends an HTTP response (booking confirmation, ticket ID, or an error if seats were unavailable) back to the browser, which displays the confirmation page to the user.